

Pilhas e Filas

Estruturas de Dados e Algoritmos

Prof. Sérgio Gorender

Introdução

- Pilhas e filas são tipos especiais de listas lineares nos quais as inserções e remoções sempre ocorrem nas extremidades da lista, e desta forma não é preciso movimentar nós.
- São chamadas de listas lineares com políticas de acesso.
- Utilizamos variáveis indicadores especiais, que funcionam como ponteiros, indicando as extremidades da lista.

Pilha - *Stack*

- Lista linear na qual as inserções e remoções são realizadas em uma mesma extremidade, chamada topo.
- Política – o último a entrar é o primeiro a sair - **LIFO** – **Last in, first out**.
- A variável Topo é uma variável inteira e sempre indica o índice do elemento da lista no topo.
- Operações:
 - Inserção – empilhar – **Push** - a cada inserção o Topo é incrementado.
 - Remoção – desempilhar – **Pop** - a cada remoção o Topo é decrementado.

Pilha – algoritmo de inserção - push

- A constante **M** indica o tamanho do vetor alocado para a pilha (representa o tamanho da memória previamente alocada).
- Ao iniciar a pilha, ela está vazia, e a variável **topo** será igual a **0**.

Function Empilhar(pilha, topo, valor)

Complexidade $O(1)$

if topo < M **then**

 topo = topo + 1

 pilha[topo] = valor

else *overflow*

return pilha, topo

Pilha – algoritmo de inserção - push

```
lista = [0,0,0,0,0,0,0,0,0,0]
topo = -1;
m = 9;
```

Em Python:

```
def empilhar(chave,topo, m):
    if topo<m:
        topo = topo + 1;
        lista[topo] = chave
    else:
        print("Overflow")
    return topo
```

Pilha – algoritmo de remoção - pop

- Quando topo for igual a 0, indica que a pilha está vazia.

```
Function Desempilhar(pilha, topo)
    if topo > 0 then
        valor = pilha[topo]
        topo = topo - 1
    else underflow
    return pilha, topo, valor
```

- A complexidade $O(1)$.

Pilha – algoritmo de remoção - pop

```
def desempilhar(topo):  
    chave = 0  
    if (topo > -1):  
        chave = lista[topo]  
        topo = topo - 1  
        print(chave);  
    else:  
        print("Underflow")  
    return topo
```

Exercício

- Escrever um algoritmo utilizando uma pilha para reconhecer se uma palavra em uma variável do tipo string é um palíndromo.
 - Uma palavra é um palíndromo se a sequência de letras que a forma é a mesma, quer seja lendo da esquerda para a direita ou da direita para a esquerda.

Fila - Queue

- Fila é uma lista linear na qual as inserções são realizadas em uma extremidade e as remoções na outra.
- As remoções são realizadas no início da fila e as inserções em sua retaguarda.
- Política – primeiro a entrar é o primeiro a sair – **FIFO** – first in, first out.

Fila - Queue

- Utilizamos duas variáveis especiais, do tipo **int** para apontar para estas posições: **inicio** para indicar o início da fila e **fim** para indicar a retaguarda.
- Operações:
 - Inserção – enfileirar – **enqueue** – a variável **fim** é incrementada;
 - Remoção – desenfileirar – **dequeue** – a variável **inicio** é incrementada.

Fila – inserção - enqueue

- Enfileirar - A inserção é realizada no final da fila, indicada pela variável **fim**. Como o vetor possui um tamanho alocado de forma estática (**M**), quando **fim = M** não teremos como inserir novos elementos.

```
function Enfileirar(fila, fim, valor)
    if fim < M then
        fim = fim + 1
        fila[fim] = valor
    else overflow
    return fila, fim
```

```
function Criar-fila
    fim = 0
    inicio = 1
```

Fila – remoção - dequeue

- Desenfileirar - a remoção é realizada no início da fila, indicada pela variável **início**.

Function Desenfileirar(fila, início)

```
    if início <= fim then  
        valor = fila[início]  
        início = início + 1  
else underflow  
return fila, início, valor
```

Fila circular

- Para evitar que o deslocamento da fila no vetor, com inserções e remoções, faça a fila chegar ao final do vetor, dando uma impressão falsa de que a fila está cheia, os algoritmos de inserção e remoção de fila em alocação sequencial tratam a alocação com sendo uma fila circular.
- Quando a fila estiver vazia, devemos fazer **inicio = fim = 0**.

Fila – inserção - enqueue

- A inserção é realizada no final da fila, indicada pela variável **fim**. Para que funcione como fila circular, utilizamos um teste para que quando **fim** for igual a **M**, **fim** passe a ser **1**.

```
Funcion Enfileirar-circular(fila, inicio, fim, num-elementos, valor)
    if num-elementos < M then
        se fim == M então
            fim = 1
        else
            fim = fim + 1
        fila[fim] = valor
        num-elementos = num-elementos + 1
        if inicio == 0 then
            inicio = 1
    else overflow
    return fila, inicio, fim, num-elementos
```

```
def enfileirar(chave):  
    global fim  
    global inicio  
    global num  
    global tam  
    print(fim)  
    if (num <= tam):  
        fim = fim + 1  
        if (fim > tam):  
            fim = 0  
        lista[fim] = chave;  
        if (inicio == -1):  
            inicio = fim;  
        num = num + 1  
    else:  
        print("Overflow")
```

Fila Circular Enfileirar (Enqueue) em Python

Fila – remoção - dequeue

- A remoção é realizada no início da fila, indicada pela variável **início**. Para que funcione como fila circular, quando **início** for igual a **M**, para que **início** passe a ser **1**, utilizamos um teste.

```
Function Desenfileirar-circular(fila, inicio, fim, num-elementos)
    if num-elementos > 0 then
        valor = fila[inicio]
        if inicio == M then
            inicio = 1
        else
            inicio = inicio + 1
        num-elementos = num-elementos - 1
        if num-elementos == 0 then
            inicio = fim = 0
    then underflow
    return fila, inicio, fim, num-elementos, valor
```



```
def desenfileirar():  
    global fim  
    global inicio  
    global tam  
    global num  
    if (num > 0):  
        chave = lista[inicio]  
        inicio = inicio + 1  
        if (inicio > tam):  
            inicio = 0  
        print(chave);  
        num = num - 1  
    else:  
        print("Underflow")
```

Fila Circular
Desenfileirar
(Dequeue) em
Python

Fila – inserção - enqueue

- A estrutura circular da fila também pode ser feita utilizando a função módulo.

funcion Enfileirar-circular(fila, inicio, fim, num-elementos, valor)

if num-elementos < M **then**

 fim = fim + 1 mod m

 fila[fim] = valor

 num-elementos = num-elementos + 1

if inicio == 0 **then**

 inicio = 1

else overflow

 return fila, inicio, fim, num-elementos

Fila – remoção - dequeue

- A estrutura de fila circular também pode ser mantida utilizando a função módulo.

```
Function Desenfileirar-circular(fila, inicio, fim, num-elementos)
    if num-elementos > 0 then
        valor = fila[inicio]
        inicio = (inicio + 1) mod M
        num-elementos = num-elementos – 1
        if num-elementos == 0 then
            inicio = fim = 0
    then underflow
    return fila, inicio, fim, num-elementos, valor
```

Notação Polonesa

- A notação polonesa é um formato utilizado para tornar claras e sem ambiguidades expressões aritméticas, para que possam ser resolvidas computacionalmente.
- É utilizada em compiladores.
- Em uma expressão aritmética na notação polonesa os operadores aparecem imediatamente antes dos operandos, dispensando o uso de parênteses.
- Notação Polonesa reversa – Operadores aparecem após os operandos, na ordem em que devem ser calculados.

Notação Polonesa

- Ex:
 - $A * B - C / D$;
 - Notação parentizada: $((A*B)-(C/D))$
 - Em notação polonesa: $- * A B / C D$.
 - Notação polonesa reversa: $A B * C D / -$
- $A * B / C + E * F$;
- Notação parentizada: $((A*B)/C)+(E*F)$
- Em notação polonesa: $+ / * A B C * E F$.
- Notação polonesa reversa: $A B * C / E F * +$

```

indexp = 1; indpol = 0; topo = 0
while indexp <= fim do
    if exp[indexp] é operando then
        indpol = indpol + 1
        pol[indpol] = exp[indexp]
    else if exp[indexp] é operador then
        topo = topo + 1
        pilha[topo] = exp[indexp]
    else if exp[indexp] = ")" then
        if topo != 0 then
            operador = pilha[topo]
            topo = topo - 1
            indpol = indpol + 1
            pol[indpol] = operador
        else "expressão errada"

    indexp = indexp + 1

```

Conversão
 de
 expressão
 para
 Notação
 Polonesa
 Reversa
 Entrada –
 expressão com
 notação
 parentizada.

Execução de expressão em Notação Polonesa Reversa

```
topo = 0
ind = 1; fim = tamexp;
while ind <= fim do
    if exp[ind] é operando then
        topo = topo + 1
        pilha[topo] = exp[ind]
    else
        op1 = pilha[topo]
        topo = topo - 1
        op2 = pilha[topo]
        operador = exp[ind]
        executa(op1, op2, operador, res)
        pilha[topo] = res
    ind = ind + 1
resultado = pilha[topo]
```